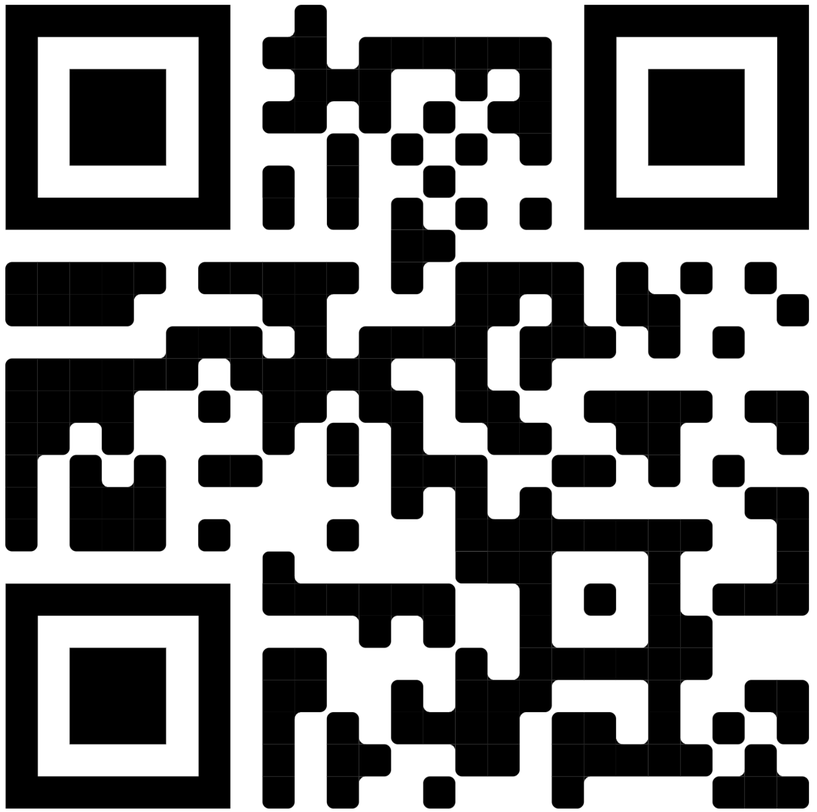


http://bit.ly/rs-tu-sam





Distributed systems

Lectures: doc. dr. sc. Nikola Tanković

Time, clocks and ordering



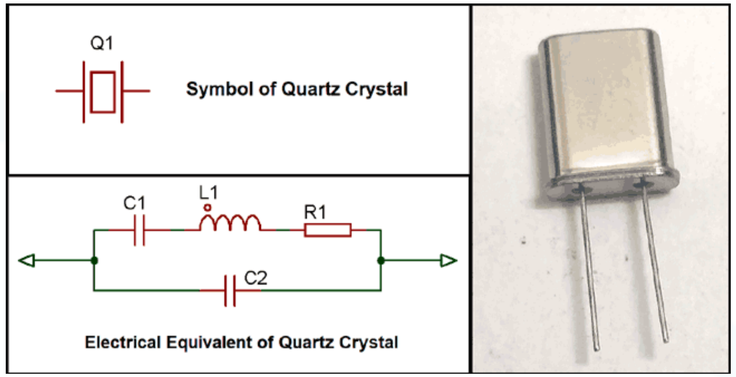
Time and clocks

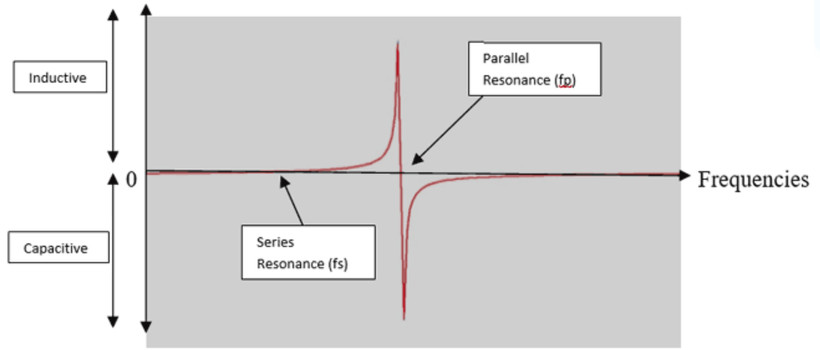
- Why do we need them?
  - Timeouts and retries,
  - Performance monitoring & modeling
  - Event logs
  - TTLs (time-to-live): cache, authentication tokens, record expiry (e.g., DynamoDB)...
  - Determining the order of events
- Types of clocks
  - **physical** - number of (milli)seconds elapsed
  - **logical** - number of events elapsed

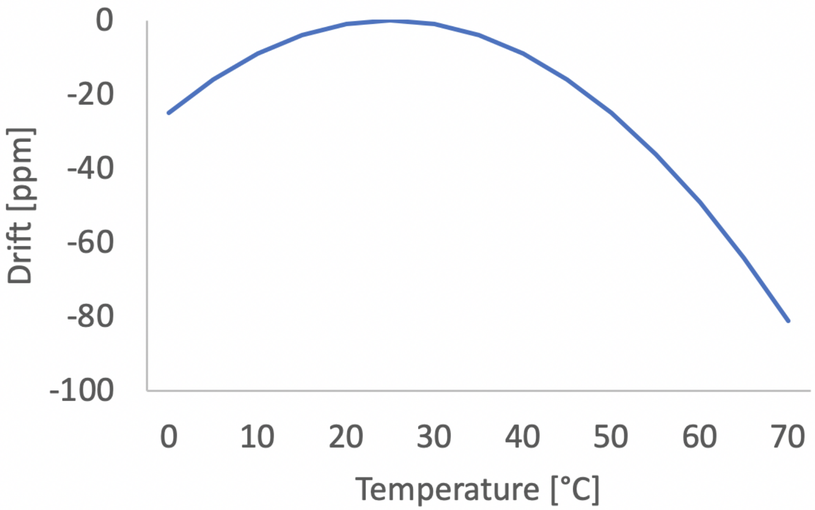
Physical clocks

- **Quartz clock**
  - Specifically designed to resonate at specific frequency
    - e.g. 32768Hz or 2^15Hz
  - Piezo-electric effect
    - "bending the voltage"
  - Cheap to produce
  - Imprecise (**1ppm** = 32s/year)



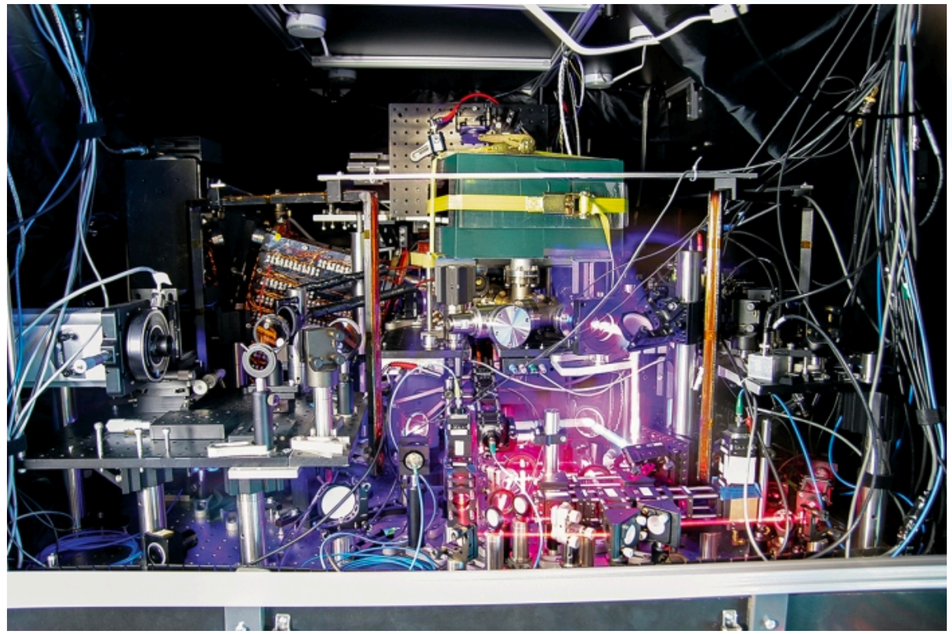






Physical clocks

- **Atomic clocks**
  - Greater precision
    - 1 second in 3 million years
  - The international system of Units (SI) definition:
    - **1 second** = 9,192,631,770 periods of a particular resonant frequency of the cesium-133 atom
  - Price ~ 20.000$



Atomic clock



Leap seconds

- **Greenwich Mean Time (GMT, solar time)**: it's noon when the sun is in the south, as seen from the Greenwich meridian
- **International Atomic Time (TAI)**: 1 day is 24 × 60 × 60 × 9,192,631,770 periods of caesium-133's resonant frequency

- **Problem**: it works great (!) but is disconnected from our feeling of time - the speed of Earth's rotation is not constant!

- **Compromise**: UTC time is the TAI time, but with the corrections applied to account for Earth's rotation

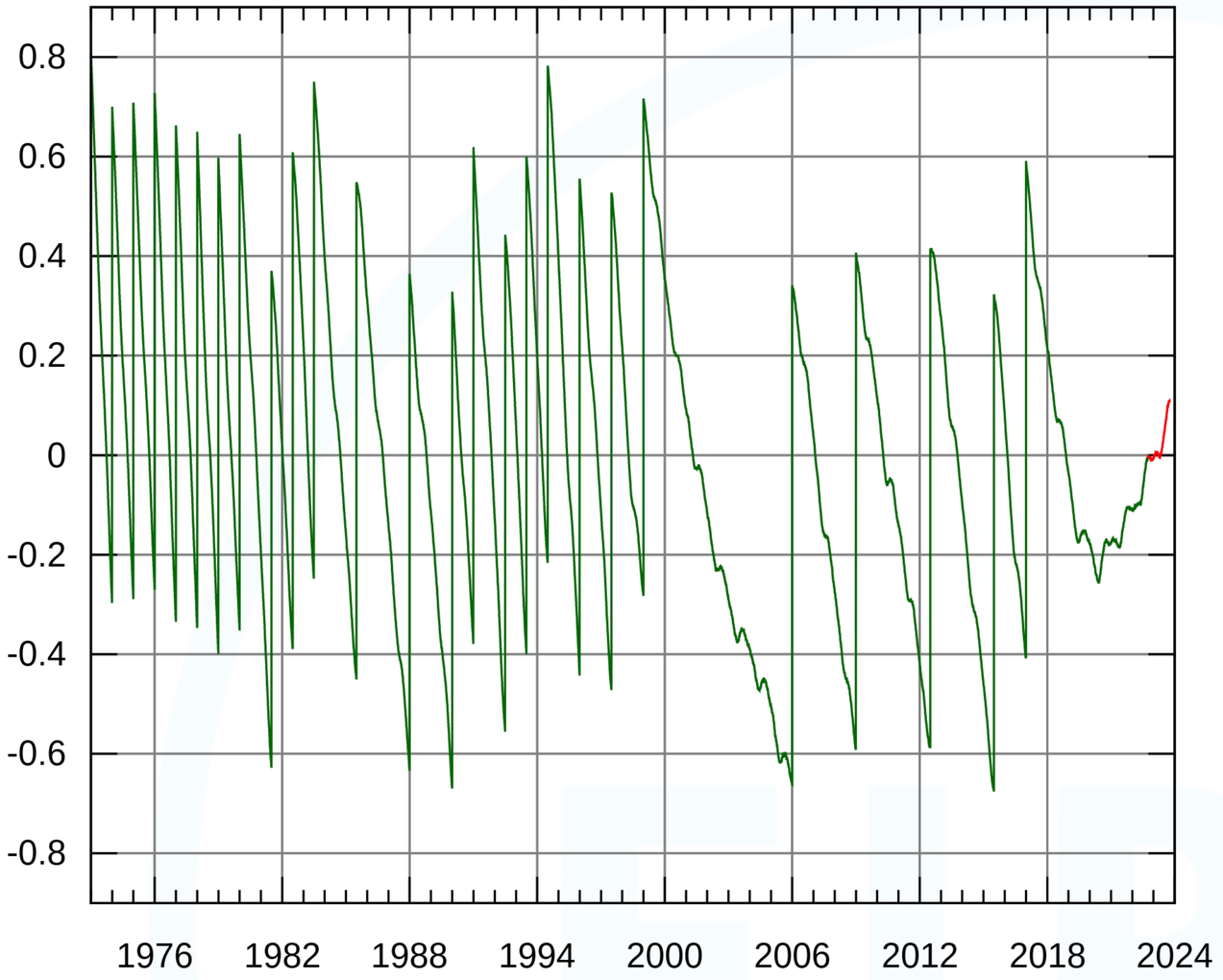
Leap seconds

- Every year, on 30 June and 31 December at 23:59:59 UTC, one of three things happens:
  - The clock immediately jumps forward to 00:00:00, skipping one second (**negative leap second**)
  - The clock moves to 00:00:00 after one second, as usual
  - The clock moves to 23:59:60 after one second, and then moves to 00:00:00 after one further second (**positive leap** second)
- This is announced several months beforehand.

Yes, this is possible



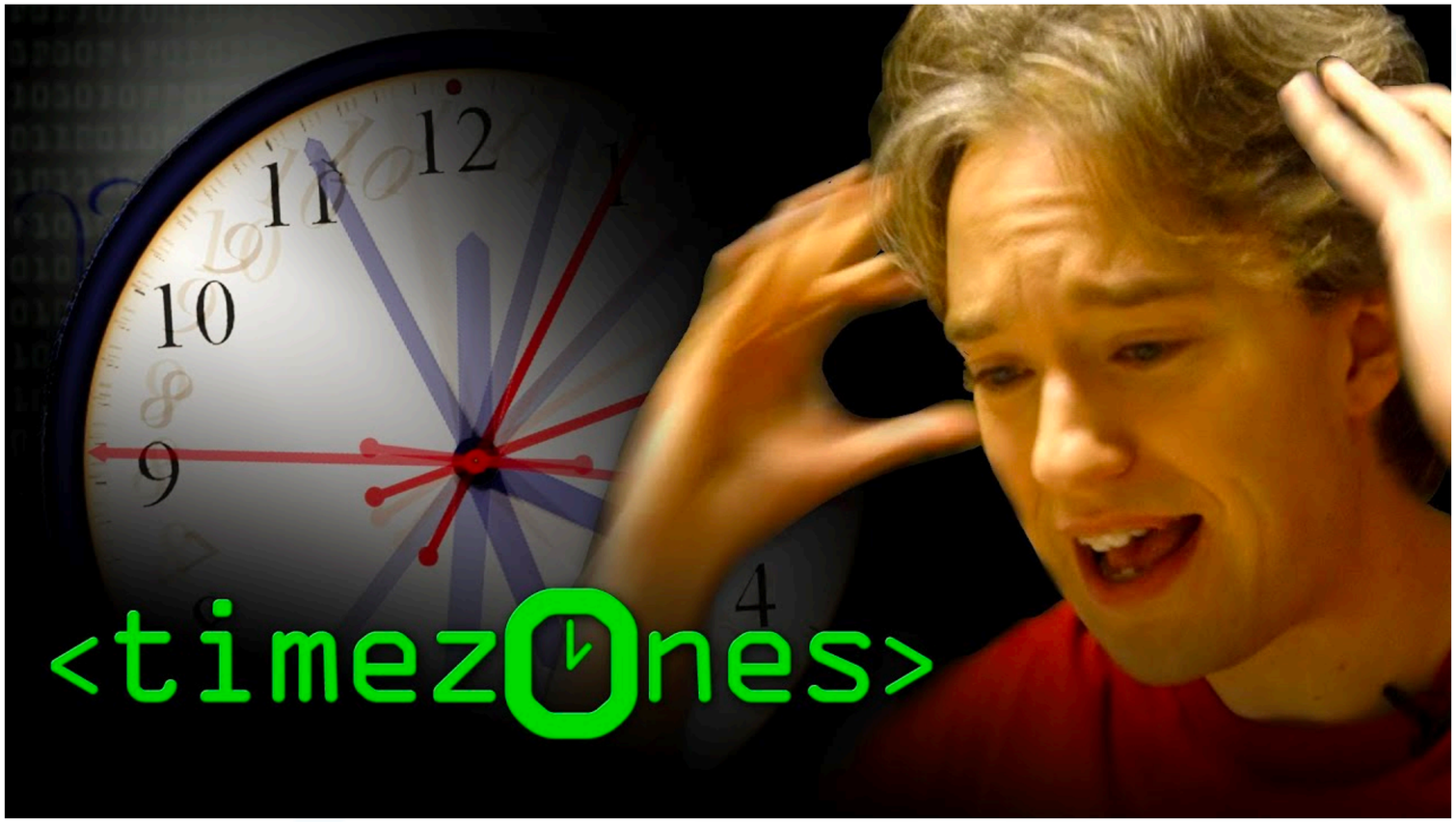
In practice...


0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
1976
1982
1988
1994
2000
2006
2012
2018
2024

Representation of time

- **How to squeeze the time into bits**
- **Unix timestamp**
  - Number of seconds elapsed from 1 January 1970 00:00:00 UTC
  - *not dealing with leap seconds*
- **ISO 8601**
  - Taking time-zones into consideration 🤯

Timezones are a mess...



https://www.youtube.com/watch?v=-5wpm-gesOY

Dealing with those leap seconds

- Most software ignores it
- But distributed systems and OS'es need microsecond precisions
  - Many failures due to leap seconds
- Pragmatic solution: **smear** that second throughout the day
  - Take or add a very small part every microsecond

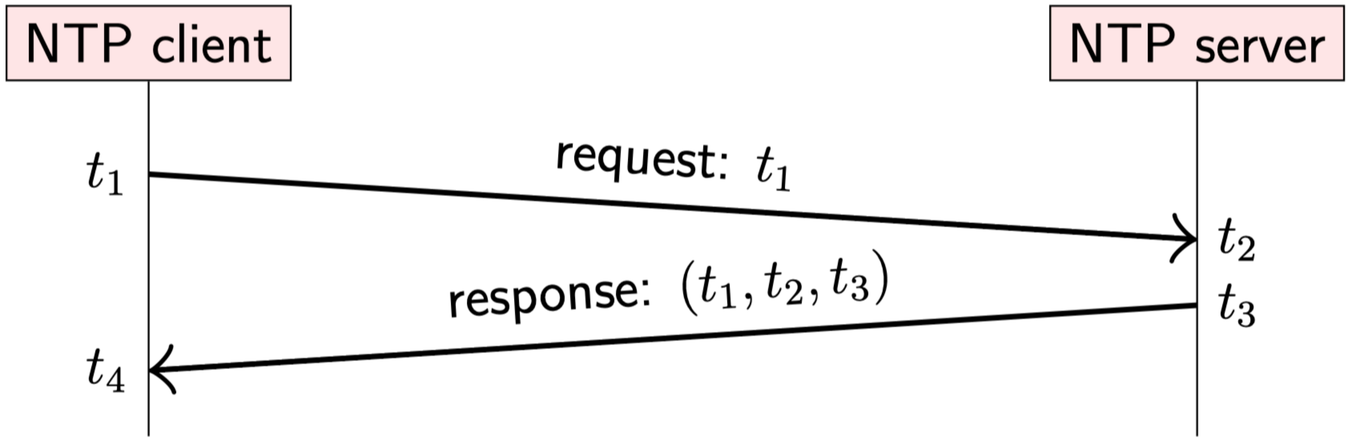
Clock synchronization

- Computers have a battery for the quartz clocks
  - due to drift (temperature et al.), the clocks continually depart from one another
- **Clock skew**: difference in clocks across systems
- **Solution:** get the time from an atomic clock or GPS (which have atomic clocks)
- **Protocol for sync:** Network Time Protocol (NTP)

Network Time Protocol

- Many vendors configure their OSes to sync time with their institutional **NTP servers**
- **National Institute** of **Standardization** and **Technology** has important NTP servers
  - https://tf.nist.gov/tf-cgi/servers.cgi
- Hierarchy (organized in **stratums**)
  - Stratum 0: atomic clock
  - Stratum 1: synced with Stratum 0
  - Stratum 2: ...
- But how to deal with **latency**?

Time synchronization



Round-trip network delay: $\delta = (t_4 - t_1) - (t_3 - t_2)$

Estimated server time when client receives response: $t_3 + \frac{\delta}{2}$

Estimated clock skew: $\theta = t_3 + \frac{\delta}{2} - t_4 = \frac{t_2 - t_1 + t_3 - t_4}{2}$

Correcting the skew

- Once the client has estimated the clock skew θ, it needs to apply that correction to its clock.
  - If | θ | < 125 ms, **slew** the clock: slightly speed it up or slow it down by up to 500 ppm (brings clocks in sync within ≈ 5 minutes)
  - If 125 ms ≤ | θ | < 1,000 s, step the clock: suddenly reset the client clock to the estimated server timestamp
  - If | θ | ≥ 1,000 s, panic and do nothing (leave the problem for a human operator to resolve)
- Systems that rely on clock sync need to monitor clock skew!

How to measure performance

Why can the measured time be negative?



```
import time

# bad practice
start_time = time.time_ns()
call_some_rpc()
end_time = time.time_ns()

elapsed = end_time - start_time
print(f"The call took {elapsed} nanoseconds.")
```

```
# good practice
start_time = time.monotonic_ns()
call_some_rpc()
end_time = time.monotonic_ns()

elapsed = end_time - start_time
print(f"The call took {elapsed} nanoseconds.")
```

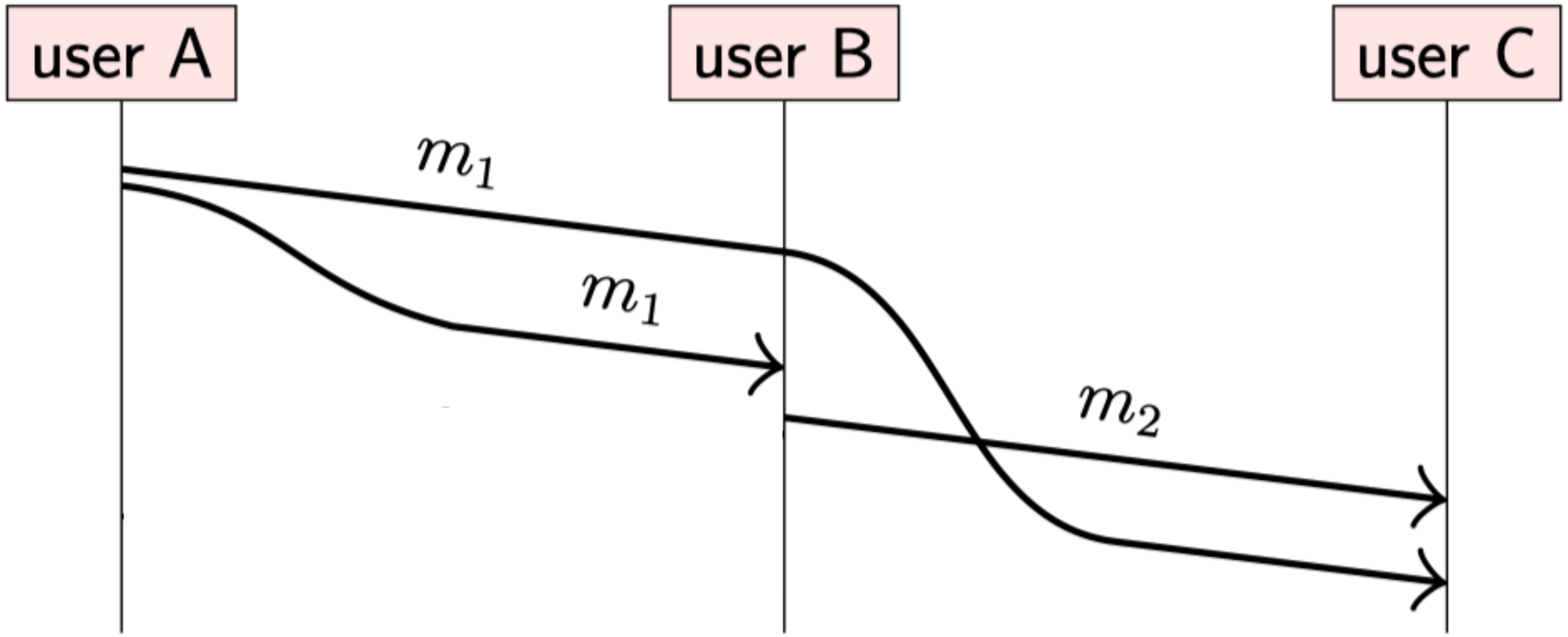




Distributed systems

Ordering

Ordering the messages



- m_1 = "I was first!"
- m_2 = "I was first!"

Author: Dr. Martin Kleppmann

Inconsistencies

- Clocks are skewing
- Messages are arbitrarily late
- What can be done?



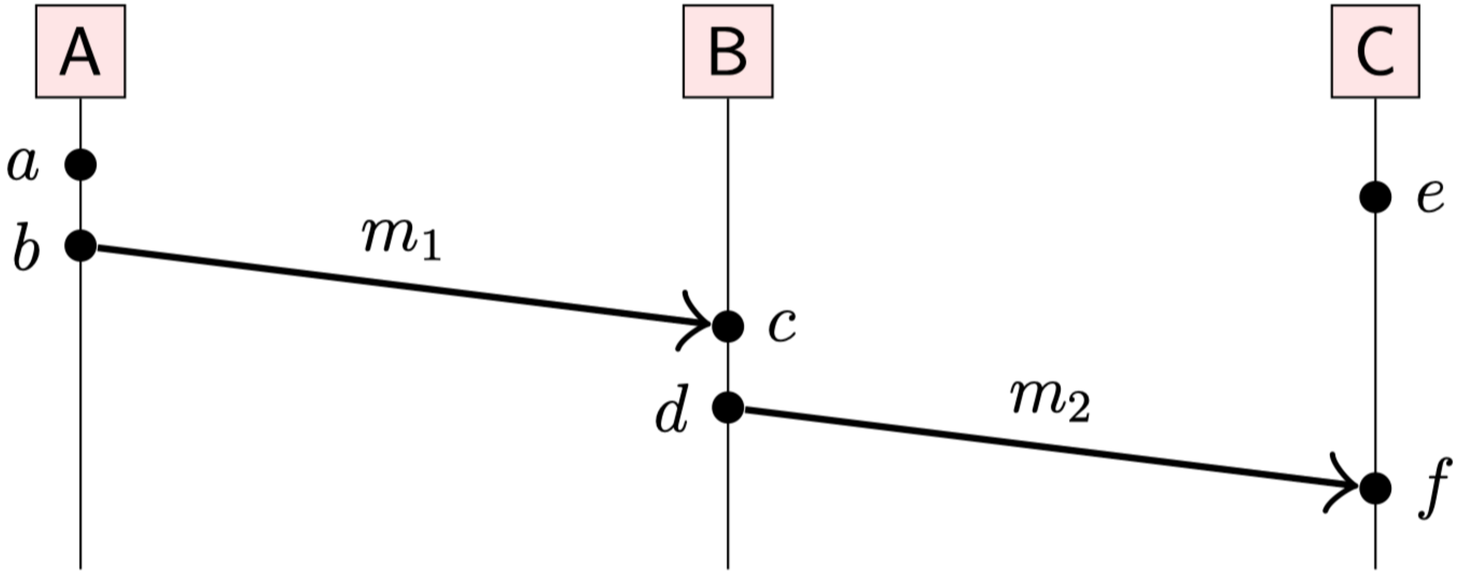


DEF

"Happens-before" partial order

- An **event** is a unique moment in time at certain node
- We can say that ***a happens before b,*** noted with $a \rightarrow b$ if and only if:
  - Both a and b happened on the same node, and a before b
  - Event a is the event of sending some message, and event b is the receiving it
  - If c exists where $a \rightarrow c \wedge c \rightarrow b$ (transitivity)

    $\exists c : a \rightarrow c \wedge c \rightarrow b \implies a \rightarrow b$
  - Otherwise we can note $a \parallel b$
    - a **is concurrent with** b

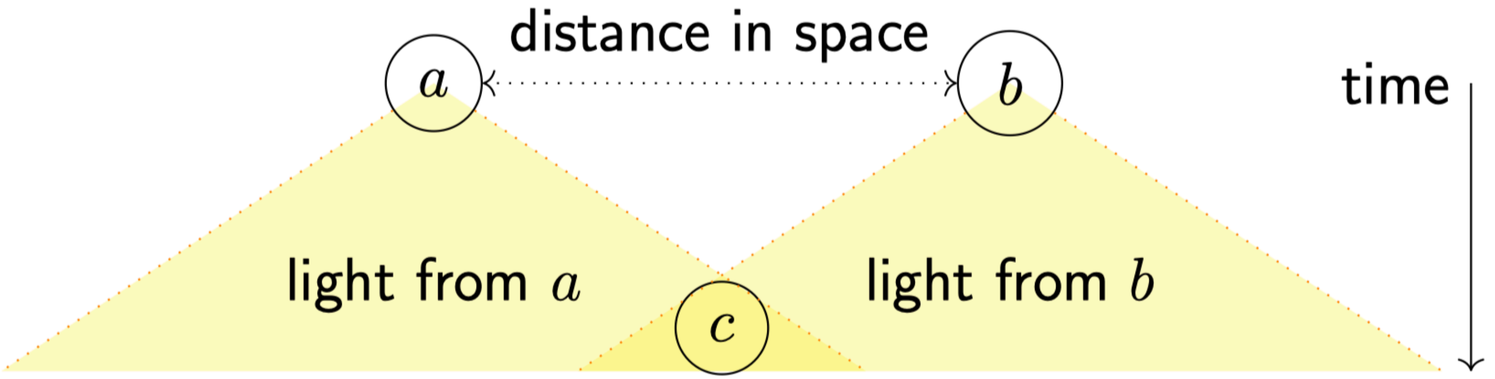


- $a \rightarrow b$, $c \rightarrow d$, and $e \rightarrow f$ due to node execution order
- $b \rightarrow c$ and $d \rightarrow f$ due to messages m_1 and m_2
- $a \rightarrow c$, $a \rightarrow d$, $a \rightarrow f$, $b \rightarrow d$, $b \rightarrow f$, and $c \rightarrow f$ due to transitivity
- $a \parallel e$, $b \parallel e$, $c \parallel e$, and $d \parallel e$

Author: Dr. Martin Kleppmann

Causality

- Borrowed from physics (relativity) - based on information flow
  - When $a \rightarrow b$ then a **might have caused** b
  - When $a \| b$ then a **did not have caused** b



Let $\prec$ be a strict total order on events.
If $(a \rightarrow b) \Longrightarrow (a \prec b)$ then $\prec$ is a **causal order**
(or: $\prec$ is "consistent with causality").

Finally: Logical clocks

- Recap:
  - **Physical clock** - count the number of **seconds elapsed**
  - **Logical clock** - count the number of events occured
- Physical timestamps are **inconsistent with causality** (remember why?)
- Logical clocks are designed **to capture it**

$$e_1 \rightarrow e_2 \implies T(e_1) < T(e_2)$$

Lamport clocks



- Leslie Lamport (1941-)
- Paxos (consensus), bakery algorithm (concurrent safety)
- Famous papers:
  - Byzantine Generals
  - Time, Clocks, and the Ordering of Events in a distributed system
- LaTeX, temporal logic (TLA)
  - 2013 Turing award

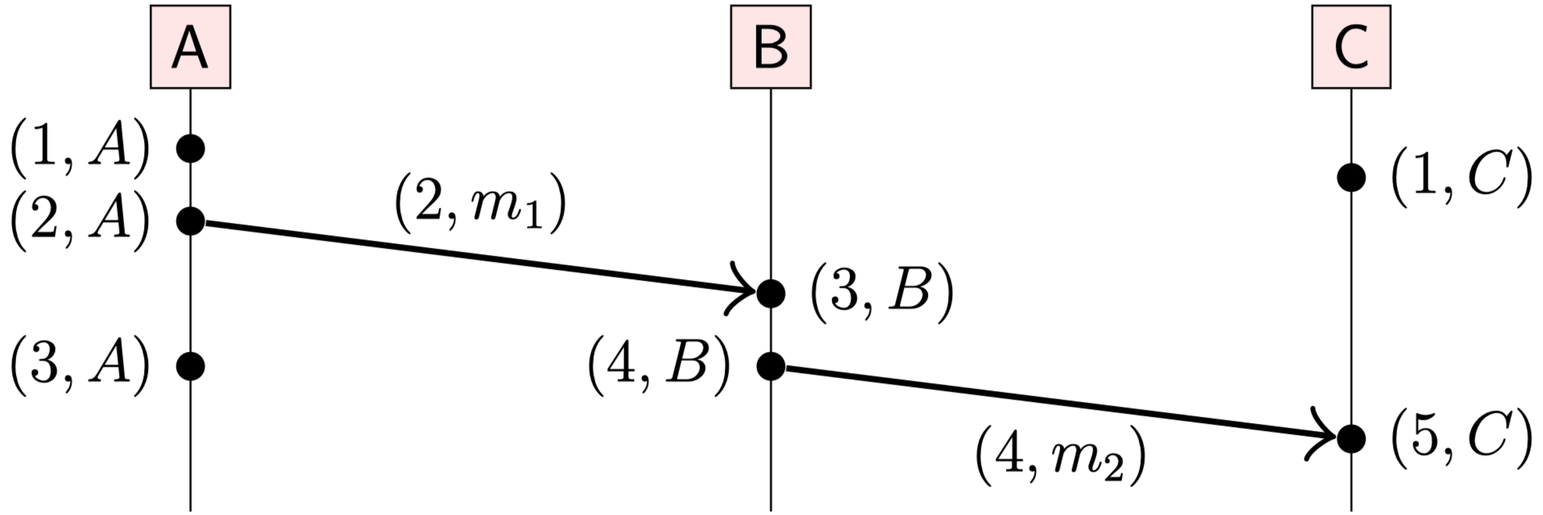
Lamport clock example

- Each node maintains a counter t (timestamp) and increments it on every event e
- After increment, that is the new Lamport timestamp $L(e)$ of that event
- If you send a message, attach the current t (timestamp) to it
- If the recipient receives a larger timestamp, it will increment its local timestamp
- Properties you get

  $a \rightarrow b \Longrightarrow L(a) < L(b)$ (but not vice-versa)

  possible that $L(a) = L(b)$ for $a \neq b$

Example



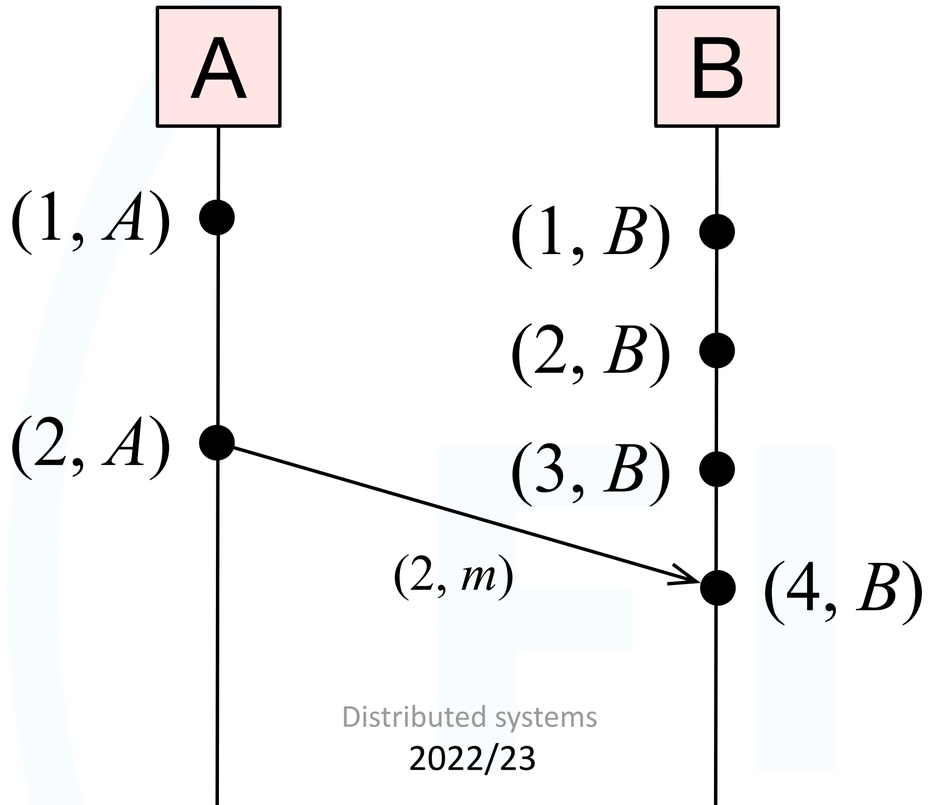
Let $N(e)$ be the **node label** in which event e occured. Pair $(L(e), N(e))$ **uniquely defines** event e.
We can break ties with a **lexicographic** order to get a **total** order.

$$a \prec b \overset{def}{\iff} (L(a) < L(b)) \vee (L(a) = L(b) \wedge (N(a) < N(b))$$

This also is a **causal** order, whenever $a \rightarrow b$ we also have $a \prec b$

Limitations

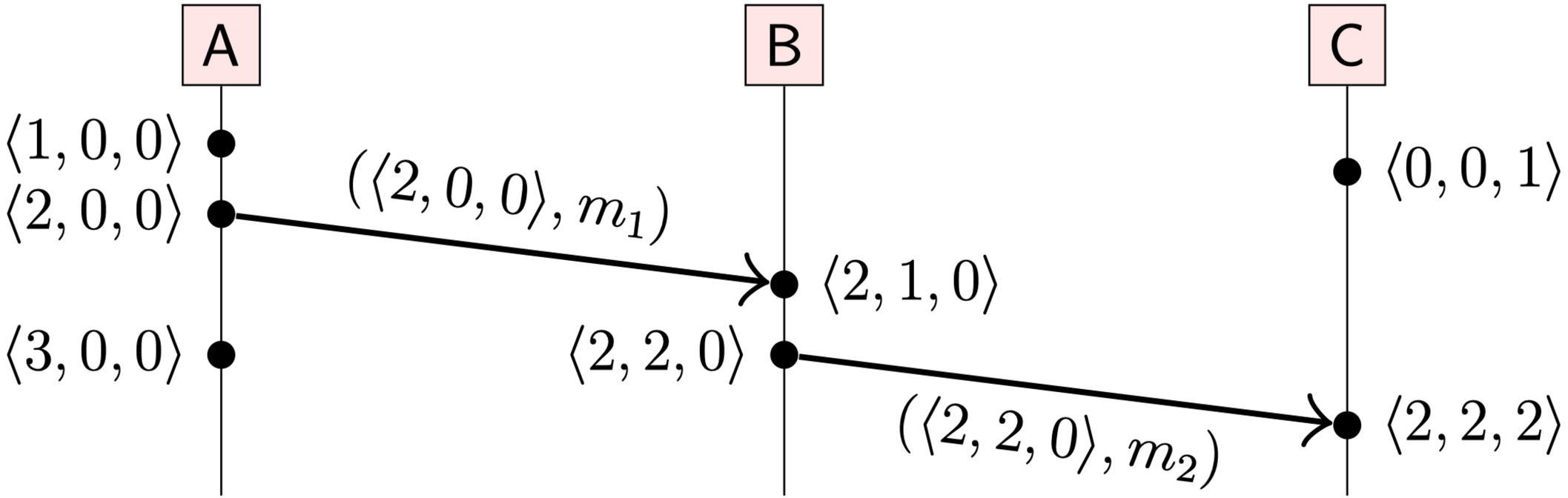
- In general, it is not possible to tell if two events are **concurrent** or if one **happened before**
- E.g. we cannot tell much about the relation between $(2,\ A)$ and $(3,\ B)$



Vector clocks

- Given two Lamport timestamps $L(a)$ and $L(b)$ we cannot say whether $a \parallel b$ or $a \rightarrow b$
- For detecting concurrent events, we need **vector clocks**
  - Assume n nodes in the system, $N = \{N_1, N_2, \ldots, N_n\}$
  - Vector timestamp of event a is $V(A) = \{t_1, t_2, \ldots, t_n\}$ where t_i is number of events observed by node N_i (incremented on each local event)
  - Each nodes maintains current vector timestamp T which it attaches to any message
  - Recipient merges the incoming vector into its local vector with element-wise **max** operation $T[j] := \max(T[j], T'[j]) \forall j \in \{1,...,n\}$

Example



Vector clocks **ordering**

Define the following order on vector timestamps (in a system with n nodes):

- $T = T'$ iff $T[i] = T'[i]$ for all $i \in \{1, \ldots, n\}$ → $\langle 1,2,3 \rangle = \langle 1,2,3 \rangle$
- $T \leq T'$ iff $T[i] \leq T'[i]$ for all $i \in \{1, \ldots, n\}$ → $\langle 1,2,2 \rangle \leq \langle 1,2,3 \rangle$ $\langle 1,2,3 \rangle \leq \langle 1,2,3 \rangle$
- $T < T'$ iff $T \leq T'$ and $T \neq T'$ → $\langle 1,2,2 \rangle < \langle 1,2,3 \rangle$
- $T \parallel T'$ iff $T \not\leq T'$ and $T' \not\leq T$ → $\langle 1,3,2 \rangle \parallel \langle 1,2,3 \rangle$

$V(a) \leq V(b)$ iff $(\{a\} \cup \{e \mid e \rightarrow a\}) \subseteq (\{b\} \cup \{e \mid e \rightarrow b\})$

Event a together with its own "happened-before" events

Properties of this order:

- $(V(a) < V(b)) \iff (a \rightarrow b)$
- $(V(a) = V(b)) \iff (a = b)$
- $(V(a) \parallel V(b)) \iff (a \parallel b)$

Self-assesment

Why do we need to keep track of the time between the nodes?

What types of clocks are there? What is the main difference between them?

What is the difference between quartz and an atomic clock? Which costs much more?

What benefits does the "smearing the leap second" bring?

Define a Lamport clock. What are its down-sides?

Define a Vector clock, and compare it to the characteristics of the Lamport clock.